

	Departamento De Ciencias de la Computación
	Carrera: Ingeniería en Tecnologías de la Información Asignatura: Metodología de Desarrollo de Software NRC: 30693
	Apellidos y nombres del estudiante: JARAMILLO APUNTE CHRISTOPHER MATEO

ANTECEDENTES

1. Este taller se basa en la necesidad de orientar el desarrollo de diagramas de casos de uso de análisis utilizando PlantUML, partiendo de requisitos funcionales de nivel 0.
2. Se toma como referencia una secuencia académica progresiva que conecta la ERS/SRS con el modelado estructural y dinámico.
3. El ejercicio práctico se centra en la gestión de un CRUD de Estudiantes (Agregar, Actualizar, Eliminar y Mostrar Todo).

OBJETIVOS

Objetivo General:

Desarrollar un modelo de análisis para un sistema CRUD de gestión de estudiantes, aplicando UML para representar la interacción entre la lógica de negocio en Java y la persistencia de datos en archivos planos, garantizando la trazabilidad entre los requisitos funcionales y la implementación técnica.

Objetivos Específicos:

1. Desarrollar diagramas de casos de uso claros, trazables y verificables en PlantUML a partir de requisitos funcionales.
2. Mantener la trazabilidad entre el requisito, el actor, el caso de uso y el criterio de aceptación mediante una matriz RF-CU.
3. Identificar los componentes base para la transición hacia el modelado estructural de clases (frontera, control y entidad).

DESARROLLO

1. Definición de Requisitos Funcionales

Se han identificado 5 requisitos funcionales principales para el sistema de gestión de estudiantes:

Código	Requisito funcional de nivel 0	Actor principal	Caso de uso asociado
--------	--------------------------------	-----------------	----------------------

RF-01	El sistema debe permitir agregar un nuevo estudiante con ID, Nombre y Edad.	Administrador académico	Agregar estudiante
RF-02	El sistema debe permitir actualizar los datos de un estudiante existente usando ID, Nombre y Edad.	Administrador académico	Actualizar estudiante
RF-03	El sistema debe permitir eliminar un estudiante seleccionado cuando corresponda según la regla académica definida.	Administrador académico	Eliminar estudiante
RF-04	El sistema debe permitir mostrar todos los estudiantes registrados en una tabla con las columnas ID, Nombre y Edad.	Administrador académico	Mostrar todo
RF-05	El sistema debe validar los datos obligatorios antes de guardar o actualizar cambios.	Sistema	Validar datos del estudiante

2. Modelado en PlantUML

Diagrama de casos de uso CRUD Estudiante

```

@startuml
' --- CONFIGURACIÓN DE ESTILO ---
left to right direction
skinparam packageStyle rectangle
skinparam shadowing false
skinparam roundCorner 10

' Estilo de Actores
skinparam actor {
    BackgroundColor #E1F5FE
    BorderColor #01579B
}

```

```

' Estilo General de Casos de Uso
skinparam usecase {
    BackgroundColor #FFFFFF
    BorderColor #2C3E50
}

' --- ELEMENTOS ---

actor "Estudiante" as Admin

rectangle "Sistema CRUD de Estudiantes" #F8F9F9 {

    usecase "Agregar estudiante" as UC1 #D4EFDf
    usecase "Actualizar estudiante" as UC2 #D6EAF8
    usecase "Eliminar estudiante" as UC3 #FADBD8
    usecase "Mostrar todo" as UC4 #FEF9E7

    ' Caso de uso de validación (soporte)
    usecase "Validar datos del estudiante" as UC5 #E8DAEF
}

' --- RELACIONES ---

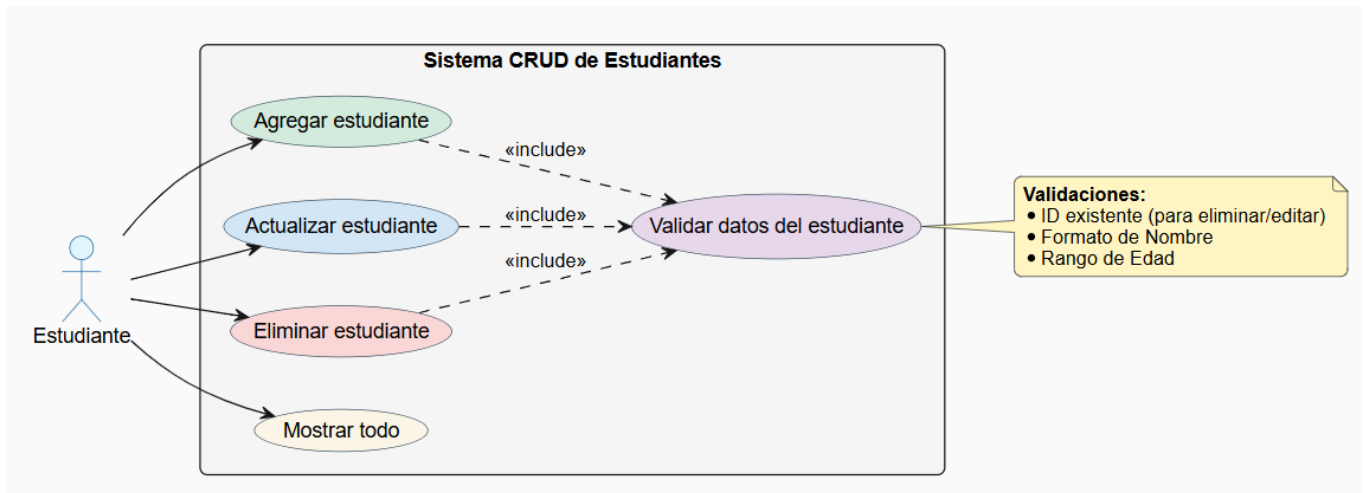
Admin --> UC1
Admin --> UC2
Admin --> UC3
Admin --> UC4

' Relaciones de INCLUSIÓN (incluyendo ahora Eliminar)
UC1 ..> UC5 : <<include>>
UC2 ..> UC5 : <<include>>
UC3 ..> UC5 : <<include>>

' --- NOTA ---
note right of UC5 #FFF9C4
<B>Validaciones:</B>
* ID existente (para eliminar/editar)
* Formato de Nombre
* Rango de Edad
end note

@enduml

```



AGREGAR ESTUDIANTE

```
@startuml
title Proceso: Agregar Estudiante (Modelo.agregar)
left to right direction

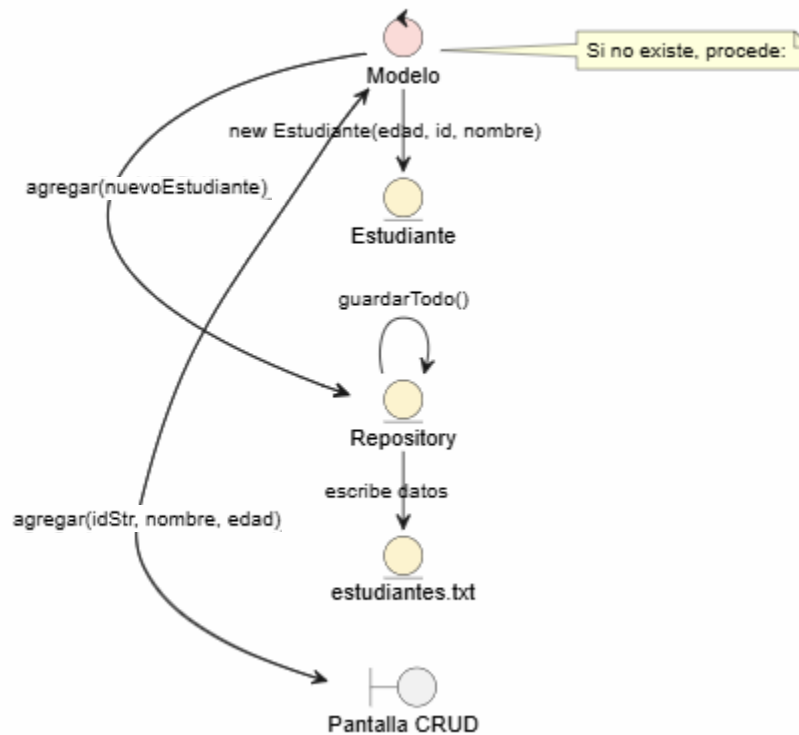
skinparam {
    ControlBackgroundColor #FADBD8
    EntityBackgroundColor #FCF3CF
}

boundary "Pantalla CRUD" as UI
control "Modelo" as Ctrl
entity "Repository" as Repo
entity "Estudiante" as Est
entity "estudiantes.txt" as File

UI -> Ctrl : agregar(idStr, nombre, edad)
Ctrl -> Repo : existId(idStr)
note right of Ctrl : Si no existe, procede:

Ctrl -> Est : new Estudiante(edad, id, nombre)
Ctrl -> Repo : agregar(nuevoEstudiante)
Repo -> Repo : guardarTodo()
Repo -> File : escribe datos
Ctrl -> UI : return true
@enduml
```

Proceso: Agregar Estudiante (Modelo.agregar)



ACTUALIZAR ESTUDIANTE

```

@startuml
title Proceso: Actualizar Estudiante (Modelo.actualizar)
left to right direction

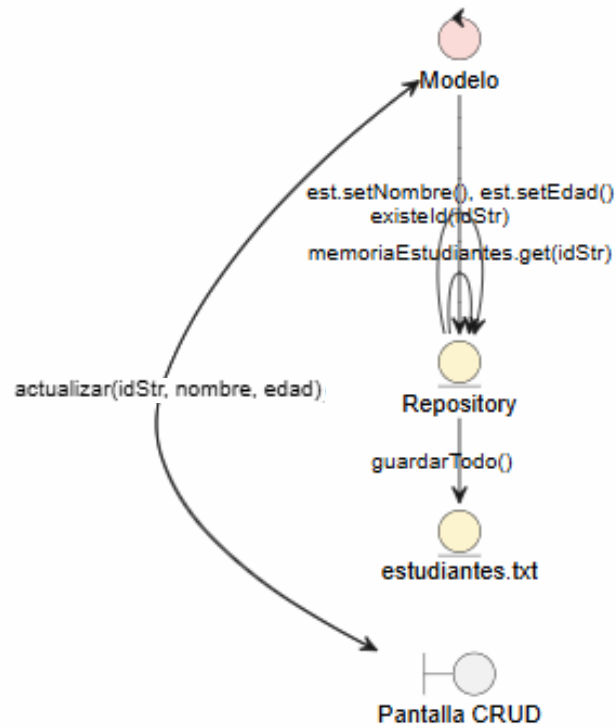
skinparam {
    ControlBackgroundColor #FADBD8
    EntityBackgroundColor #FCF3CF
}

boundary "Pantalla CRUD" as UI
control "Modelo" as Ctrl
entity "Repository" as Repo
entity "estudiantes.txt" as File

UI -> Ctrl : actualizar(idStr, nombre, edad)
Ctrl -> Repo : existeld(idStr)
Repo -> Repo : memoriaEstudiantes.get(idStr)
Repo -> Repo : est.setNombre(), est.setEdad()
Repo -> File : guardarTodo()
Ctrl -> UI : return true
@enduml

```

Proceso: Actualizar Estudiante (Modelo.actualizar)



ELIMINAR ESTUDIANTE

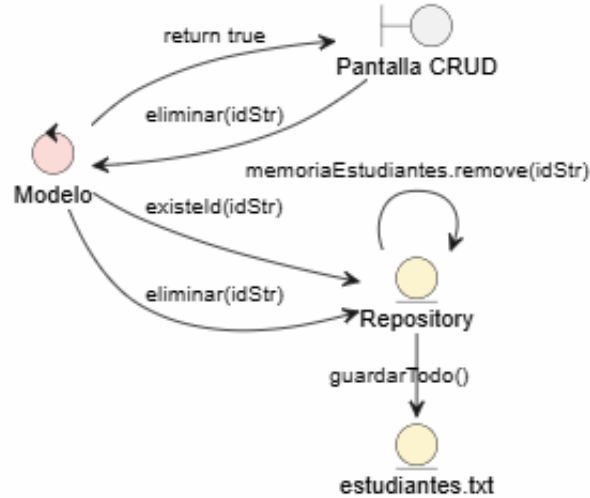
```
@startuml
title Proceso: Eliminar Estudiante (Modelo.eliminar)
left to right direction

skinparam {
    ControlBackgroundColor #FADBD8
    EntityBackgroundColor #FCF3CF
}

boundary "Pantalla CRUD" as UI
control "Modelo" as Ctrl
entity "Repository" as Repo
entity "estudiantes.txt" as File

UI --> Ctrl : eliminar(idStr)
Ctrl -> Repo : existId(idStr)
Ctrl --> Repo : eliminar(idStr)
Repo -> Repo : memoriaEstudiantes.remove(idStr)
Repo -> File : guardarTodo()
Ctrl -> UI : return true
@enduml
```

Proceso: Eliminar Estudiante (Modelo.eliminar)



MOSTRAR TODO

```

@startuml
title Proceso: Mostrar y Cargar Datos
left to right direction

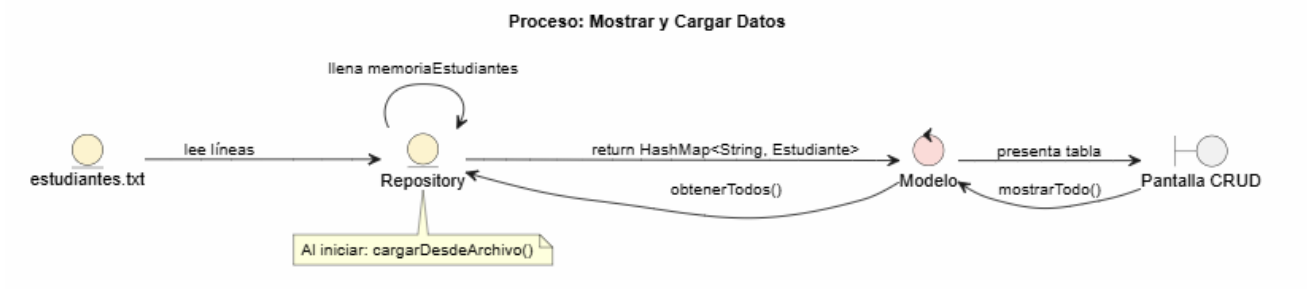
skinparam {
    ControlBackgroundColor #FADBD8
    EntityBackgroundColor #FCF3CF
}

boundary "Pantalla CRUD" as UI
control "Modelo" as Ctrl
entity "Repository" as Repo
entity "estudiantes.txt" as File

note bottom of Repo : Al iniciar: cargarDesdeArchivo()
File --> Repo : lee líneas
Repo -> Repo : llena memoriaEstudiantes

UI -> Ctrl : mostrarTodo()
Ctrl -> Repo : obtenerTodos()
Repo --> Ctrl : return HashMap<String, Estudiante>
Ctrl --> UI : presenta tabla
@enduml

```



9. Puente hacia clases de análisis: frontera, control y entidad

@startuml
left to right direction

```

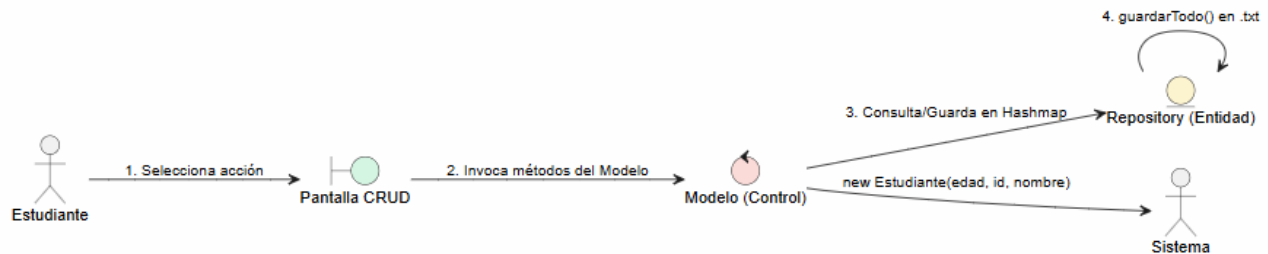
skinparam {
    BoundaryBackgroundColor #D5F5E3
    ControlBackgroundColor #FADBD8
    EntityBackgroundColor #FCF3CF
}
  
```

```

actor "Estudiante" as Admin
boundary "Pantalla CRUD" as UI
control "Modelo (Control)" as Ctrl
entity "Repository (Entidad)" as Repo
  
```

Admin --> UI : 1. Selecciona acción
 UI --> Ctrl : 2. Invoca métodos del Modelo
 Ctrl --> Repo : 3. Consulta/Guarda en Hashmap
 Repo --> Repo : 4. guardarTodo() en .txt
 Ctrl --> Sistema : new Estudiante(edad, id, nombre)

@enduml



CONCLUSIONES

1. Se concluye que usar la plataforma **PlantUML** como una herramienta de diseño mejora la creación de diagramas bajo estándares internacionales. Su enfoque basado en el código que se realizó permite una muy buena sincronización entre el diseño y los requisitos, garantizando de esta manera que la documentación técnica mejore a la par del desarrollo.
2. Como se evidenció en el proyecto, el diagrama ayudó a visualizar errores de lógica en la interacción con el archivo, previniendo fallos en la persistencia de datos.

RECOMENDACIONES

1. Se recomienda asegurarse que los nombres de los casos de uso sigan la estructura para mantener la orientación a objetivos del usuario.
2. Utilizar el diagrama de casos de uso como insumo directo para la identificación de clases de frontera, control y entidad en las fases posteriores del análisis.

N°	INTEGRANTES
1	Jaramillo Apunte Christopher Mateo
2	Sosa Morales Melany Valeria
3	Velasquez Alay Mery Stephanie
4	Verdezoto Armijos Miguel Alejandro

Sangolquí, a 22 de Mayo de 2026